Chunk-wise Model-based Policy Optimization

Weixi Song[1,2,3] **Donglin Wang**[3,†] **Yong-Lu Li**[2,4,†]

[1]Zhejiang University [2]Shanghai Innovation Institute [3]Westlake University
[4]Shanghai Jiao Tong University [†]Corresponding Author

Abstract: Model-based reinforcement learning has shown superior sample efficiency by leveraging the learned latent dynamics either in the form of derivative-free planning/imagined rollout training or dynamics gradient-based policy updates. While these approaches achieve comparatively fast learning, they all suffer, to varying degrees, from issues of recurrent compounding errors, high inference-time latency, and instability in backward propagation through time (BPTT). In this work, we propose a model-based RL approach that uses the chunk world model as a differentiable training-time evaluator to optimize the chunk actor without recurrent model invocation. Through the collaborative chunk-wise prediction of the world model and policy, we can generate H-step hypothetical rollouts in a single forward pass and directly optimize value expansion, avoiding the costly and unstable BPTT. We evaluate our method on 19 challenging high-dimensional humanoid tasks from HumanoidBench and deploy our method to the real-world for quadruped bipedal walking. Project Website.

Keywords: Model-based RL, Online RL, Continuous Control

1 Introduction

Reinforcement learning (RL) has achieved remarkable success in solving complex continuous control problems [1–3], games [4, 5] and large language model post-training [6, 7]. Modern model-free algorithms [8–11] provide general-purpose optimization methods and strong asymptotic performance, but they often require a large number of environment interactions. Model-based reinforcement learning (MBRL) offers a promising alternative by learning a surrogate world model that can provide additional information beyond real environment interactions. Modern MBRL methods have shown strong sample efficiency by leveraging learned latent dynamics in different ways. Dyna-style methods leverage the learned model to generate additional transitions as synthetic training data augmentation [5, 12–16], while MPC-style methods [17–21] use the learned model for decision-time planning. Another line of MBRL methods [22–24], termed as model-gradient-based methods, exploit the learned dynamics as the differentiable simulator to backpropagate policy gradients through predicted trajectories, providing more informative updates than derivative-free policy search.

Despite the different optimization forms, these approaches share a common structural dependency: they generate hypothetical future trajectories through recurrent model unrolling, often interleaved with repeated policy queries. This recursive invocation introduces several challenges for efficient policy optimization. First, generating an H-step hypothetical trajectory requires $O(H)$ sequential model calls, which increases both training cost and, for planning-based methods, inference-time latency. Second, when policy gradients are propagated through these recurrent rollouts, optimization must rely on backpropagation through time (BPTT), which is costly and unstable over long horizons.

Action chunking, which predicts a sequence of future actions instead of a single-step action, has been widely adopted in imitation learning to improve temporal consistency and reduce compounding action errors [25–27]. Recently, this idea has also begun to be explored in reinforcement learning. In model-free RL, Q-Chunking [28] and Decoupled Q-Chunking [29] show that operating over

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

action chunks enables unbiased n-step temporal-difference backups, thereby reducing TD bootstrap error and improving policy learning stability. Beyond model-free settings, action chunks have also been incorporated into offline model-based RL. MAC [30] jointly learns a chunk policy and a long-horizon dynamics model to scale offline model-based policy learning over extended horizons. These developments suggest that action chunking can serve as a structural tool for improving temporal credit assignment and long-horizon policy optimization. However, existing chunk-based RL methods have primarily focused on reducing bootstrap in value learning, while the potential of chunk-wise modeling for model-gradient-based policy optimization remains underexplored. This naturally raises the following question: can we simultaneously apply chunk-wise modeling to both the policy and the world dynamics model to retain the benefits of model-gradient-based optimization while avoiding the pathologies caused by recurrent model calls?

To answer this question, we propose **C**hunk-wise **M**odel-based **P**olicy **O**ptimization(CMPO) that introduces a collaborative chunk-wise modeling design for single-call model-gradient-based policy updates. Instead of learning a conventional one-step latent dynamics model $\mathbf{z_{t+1}} = f(\mathbf{z_t}, \mathbf{a_t})$ and recurrently applying it over the prediction horizon, we learn a chunk world model that directly maps the current latent state and a chunk of future actions to a sequence of future latent states, i.e., $\mathbf{z_{t+1:t+H}} = f(\mathbf{z_t}, \mathbf{a_{t:t+H-1}})$. Together with a chunk-wise actor $\pi_\theta(\mathbf{z_t}) = \mathbf{a_{t:t+H-1}}$, our method generates an H-step hypothetical rollout through a single model invocation, as shown in Figure 1. The chunk world model serves as a differentiable simulator, enabling the actor to directly optimize a multi-step value expansion objective without recurrent model-policy interactions that cause unstable BPTT. At inference time, the actor predicts an action chunk but executes only the first action, preserving closed-loop responsiveness, which outperforms MPC-style methods without test-time planning. We systematically evaluate our framework across 19 challenging, high-dimensional humanoid tasks from HumanoidBench[31]. Furthermore, to demonstrate the transferability of our approach, we apply our method to quadruped bipedal walking in the real world. Our experimental results show that CMPO achieves highly efficient policy improvement while maintaining substantially low training and inference budgets, offering an efficient path forward for generalist robot reinforcement learning.

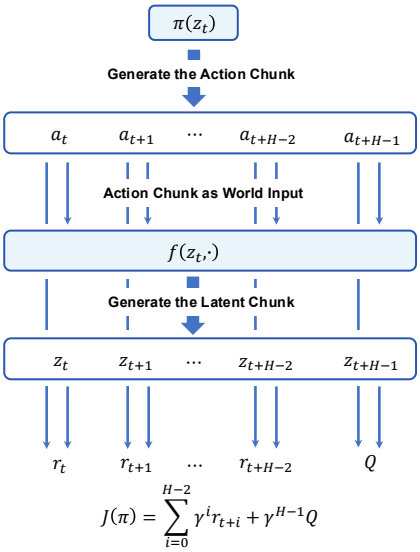


Figure 1: Computation graph of the Collaborative Chunk-wise policy update with the value expansion.

Overall, the contributions of our work are summarized as follows:

- We propose **Collaborative Chunk-wise Modeling**, which jointly models chunk-level actions and latent dynamics to generate multi-step hypothetical rollouts through a single model invocation.
- We introduce **CMPO**, a model-gradient-based RL framework that avoids recurrent model rollout and long-horizon BPTT, improving both optimization stability and computational efficiency.
- We demonstrate strong performance and sample efficiency on 19 HumanoidBench tasks, along with successful real-world deployment on quadruped bipedal locomotion.

2 Related Work

MBRL with Imagination as Augmentation. A classical way of using learned models in MBRL is to generate additional imagined experiences for policy or value learning, which can date back to Dyna [12]. Model rollouts are synthesized from real replay states and mixed with real transitions to incorporate the model-free learner [13, 32, 33]. In offline RL, methods such as MOPO [14], MOReL [34], and COMBO [35] also use model-generated rollouts to improve policy learning, and

additionally introduce uncertainty penalties, pessimistic MDPs, or conservative value regularization to mitigate out-of-distribution model exploitation. These methods treat the model primarily as a data generator and the imagined transitions or trajectories are used as additional training samples.

MBRL with Planning. Another important line of MBRL uses the learned model for decision-time planning. Early planning-based MBRL methods learn dynamics models and perform trajectory optimization through random shooting, the cross-entropy method (CEM), or related sampling-based optimizers [20, 36]. More recently, TD-MPC series [17, 18] combines short-horizon latent-space trajectory optimization with a learned terminal value function, allowing local model-based planning while relying on temporal-difference learning for long-term return estimation. Further, Wang et al. [21] bootstraps the policy learning by imitating the MPC expert and Lin et al. [37] introduces policy regularization to reduce the out-of-distribution queries by the planner and the learned policy prior.

MBRL with Model Gradient. The third category of MBRL exploits the differentiability of the learned model and leverages model gradient to directly optimize the policy. Stochastic Value Gradient(SVG) [38] provides a general framework for continuous control by backpropagating through stochastic dynamics, value functions, and policies. Amos et al. [22] combines SVG with Soft Actor Critic [8] to improve the asymptotic performance of model-based methods and Model-Augmented Actor-Critic (MAAC) [23] further derives the monotonic improvement of the pathwise derivative in terms of the gradient error in the model and the value function. Dreamer series [5, 15, 16] also optimizes the actor by propagating gradients of imagined returns through compact latent trajectories. Amigo et al. [24] instead performs trajectory unrolling with parallel simulators and propagates gradients through the learned model to mitigate compounding errors.

These categories may overlap in hybrid algorithms and our method primarily falls within the model-gradient category, while remaining orthogonal to test-time planning. In Section 4.3, we further explore the potential of integrating our method with MPC.

3 Chunk-wise Model-based Policy Optimization

3.1 Background

This work focuses on the standard reinforcement learning setting modeled as the infinite-horizon Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}(s_{t+1}|s_t, a_t)$ is the environment transition dynamics, $\mathcal{R}(s_t, a_t)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The learning objective is to optimize the actor/policy $\pi_\theta(a_t|s_t)$ that maximizes the expected cumulative discounted rewards $\mathbb{E}_\pi \sum_{t=0}^{\infty} \gamma^t r_t$.

Model-based RL (MBRL) additionally learns a surrogate world model $f_\theta(\mathbf{s_{t+1}}|\mathbf{s_t}, \mathbf{a_t})$ to approximate the environment dynamics and the reward function $R(\mathbf{s_t}, \mathbf{a_t})$ to generate immediate reward, which enables agents to simulate future trajectories, termed imagined rollouts, bypassing the need for expensive interactions with a realistic environment. Thus, an H-step value expansion with TD truncation can be formulated as:

$$\hat{G}_t^{(H)} = \sum_{k=0}^{H-1} \gamma^k R(\mathbf{z_{t+k}}, \mathbf{a_{t+k}}) + \gamma^H Q(\mathbf{z_{t+H}}, \pi(\mathbf{z_{t+H}})). \tag{1}$$

which can be utilized for test-time MPC planning [17, 18], constructing model-augmented TD targets for critic learning [39], or policy optimization [22].

3.2 Motivation

Typical model-based RL methods [5, 15–18] generate hypothetical rollouts through recurrent interactions between the world model and the actor or planner. Given a latent state z_t, a standard one-step latent dynamics model recursively predicts $\mathbf{z_{t+k+1}} = f(\mathbf{z_{t+k}}, \mathbf{a_{t+k}})$, $\mathbf{a_{t+k}} = \pi_\theta(\mathbf{z_{t+k}})$, $\mathbf{r_{t+k}} = R(\mathbf{z_{t+k}}, \mathbf{a_{t+k}})$, for $k = 0, \ldots, H-1$. The resulting imagined trajectory $(\mathbf{z}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)_{\tau=t}^{t+H}$ is then used either for policy learning, value expansion, or decision-time planning. Although this recurrent rollout design provides useful model-based signals, it also introduces several structural limitations.

First, generating an H-step hypothetical trajectory requires $\mathcal{O}(H)$ sequential model invocations. Since $\mathbf{z_{t+k+1}}$ depends on $\mathbf{z_{t+k}}$, these calls cannot be fully parallelized over the temporal dimension. This increases the training cost for imagination-based policy learning and further induces inference-time latency for MPC-style methods that repeatedly evaluate candidate action sequences.

Second, recurrent model rollouts suffer from recursive compounding prediction errors. Let $\mathbf{z_{t+k}}$ denote the ground-truth latent state and $\hat{\mathbf{z}}_{\mathbf{t+k}}$ the predicted state. If the one-step model has prediction error bounded by ϵ, and the dynamics model is L_f-Lipschitz with respect to the latent state, then the rollout error can grow as

$$\|\hat{\mathbf{z}}_{\mathbf{t+k}} - \mathbf{z_{t+k}}\| \leq \epsilon \sum_{i=0}^{k-1} L_f^i, \tag{2}$$

Proof. See Appendix A. □

which increases with the imagination horizon k, and may grow rapidly when $L_f > 1$. Consequently, the value expansion target $\hat{G}_t^{(H)}$ can become biased by errors accumulated over the entire imagined trajectory, especially for long-horizon control.

Third, for gradient-based model-based policy optimization, optimizing the policy through such recurrent hypothetical rollouts requires backward propagation through time (BPTT). The policy gradient involves products of transition Jacobians along the imagined trajectory. Specifically, the first-order gradient of future imagined states to the policy parameters θ follows

$$\frac{d\hat{\mathbf{z}}_{\mathbf{t+k+1}}}{d\theta} = \frac{\partial f}{\partial z}\frac{d\hat{\mathbf{z}}_{\mathbf{t+k}}}{d\theta} + \frac{\partial f}{\partial a}\frac{d\pi_\theta}{dz}\frac{d\hat{\mathbf{z}}_{\mathbf{t+k}}}{d\theta} = \left(\frac{\partial f}{\partial z} + \frac{\partial f}{\partial a}\frac{d\pi_\theta}{dz}\right) \cdot \frac{d\hat{\mathbf{z}}_{\mathbf{t+k}}}{d\theta}. \tag{3}$$

Unrolling this recursion yields a chain of Jacobian products across the horizon. Therefore, the gradient magnitude can be dominated by terms of the form

$$\prod_{j=i+1}^{k} \left(\frac{\partial f}{\partial z} + \frac{\partial f}{\partial a}\frac{\partial \pi_\theta}{\partial z}\right), \tag{4}$$

which may lead to exploding or vanishing gradients as k increases. In addition, BPTT requires storing the entire recurrent computation graph, further increasing memory and computational costs. We empirically verify this instability in Section 4.3, where recurrent model rollouts lead to unstable gradient norms and degraded policy updates.

Action chunking[25], as a technique widely applied in imitation learning for learning temporally consistent behaviors, has recently begun to be explored in model-free RL as well, where it has also been found to reduce temporal difference bootstrap error by using unbiased n-step backups[28, 29]. Actors on chunked action space have also been applied alongside long-horizon world models for scaling offline model-based RL in [30]. These works reveal a broader role of action chunking in reinforcement learning: it can reshape the temporal structure of policy optimization, rather than merely generating consistent low-level controls. Existing chunk-based RL methods mainly exploit this structure from the value-learning perspective. However, the same idea is particularly appealing for model-based gradient policy optimization. The key bottlenecks discussed above all stem from a step-wise rollout mechanism, where the model and the actor must be recurrently invoked to construct an H-step hypothetical trajectory.

This observation motivates our central question: can we leverage chunk-wise policy and dynamics modeling to preserve the multi-step training signal of model-based gradient optimization, while eliminating recurrent model invocation, the associated compounding error and BPTT issues?

3.3 Collaborative Chunk-wise Modeling

The core motivation is that if both the actor and the world model operate directly at the chunk level, the entire hypothetical trajectory can be generated through a single forward pass. Therefore, we

model both the policy and the world model in a chunk-wise manner. Given a current latent state $\mathbf{z_t}$, the chunked actor predicts a sequence of future actions:

$$\mathbf{a_{t:t+H-1}} = \pi(\mathbf{z_t}),$$

where H denotes the chunk horizon. Instead of using a one-step latent dynamics model $\mathbf{z_{t+1}} = f(\mathbf{z_t}, \mathbf{a_t})$ and recursively applying it H times, we train a chunk world model that directly maps the current latent state and the entire action chunk to a future latent state chunk:

$$\mathbf{z_{t+1:t+H}} = f(\mathbf{z_t}, \mathbf{a_{t:t+H-1}}).$$

This collaborative chunk-wise design changes the role of the learned world model. Rather than serving as a recurrent surrogate environment that must be rolled out step by step, the chunk world model acts as a differentiable training-time evaluator for the chunked actor. Given the predicted latent trajectory $(\mathbf{z_t}, \mathbf{a_t}, \cdots, \mathbf{a_{t+H-1}}, \mathbf{z_{t+H}})$, we can evaluate a $(H-1)$-step value expansion objective:

$$G_t^{(H-1)} = \sum_{k=0}^{H-2} \gamma^k R(\mathbf{z_{t+k}}, \mathbf{a_{t+k}}) + \gamma^{H-1} Q(\mathbf{z_{t+H-1}}, \mathbf{a_{t+H}}),$$

This formulation reduces the sequential rollout cost from $\mathcal{O}(H)$ recurrent model calls to a single chunk-level model invocation, avoids recursively compounding error mechanism of imagined latents and value estimation, and for model gradient-based policy optimization, the gradients are propagated through one chunk-level prediction that is significantly shorten by H times. The overall computational graph of the chunk-wise modeling objective is illustrated in Figure 1.

Based on the above discussion, we introduce the practical design for **C**hunk-wise **M**odel-based **P**olicy **O**ptimization(**CMPO**).

Components. We build our method upon TD-MPC2[18], a state-of-the-art model-based architecture design. Therefore, apart from the actor and the world dynamic model, the remaining design stays consistent. Specifically, it consists of the following five components:

$$\begin{array}{lll}
\text{Encoder} & \mathbf{z_t} = h(\mathbf{s_t}) & \triangleright \text{ Maps states to latent representations} \\
\text{Latent dynamics} & \mathbf{z_{t+1:t+H}} = f(\mathbf{z_t}, \mathbf{a_{t:t+H-1}}) & \triangleright \text{ Chunked latent forward dynamics} \\
\text{Reward} & \hat{\mathbf{r}}_\mathbf{t} = R(\mathbf{z_t}, \mathbf{a_t},) & \triangleright \text{ Predicts reward } r \text{ of a transition} \\
\text{Terminal value} & \hat{\mathbf{q}}_\mathbf{t} = Q(\mathbf{z_t}, \mathbf{a_t}) & \triangleright \text{ Predicts discounted sum of rewards} \\
\text{Policy prior} & \mathbf{a_{t:t+H-1}} = \pi(\mathbf{z_t}) & \triangleright \text{ Predicts chunked action}
\end{array} \tag{5}$$

Model Learning. Following the basic framework of the prior work, the training objective of the state encoder h, the chunked world dynamics model f, and the reward model R is summarized as follows:

$$\begin{aligned}
\mathcal{L}_{\text{model}} = \mathop{\mathbb{E}}_{(\mathbf{s_t}, \mathbf{a_t}, \mathbf{r_t}, \mathbf{s_{t+1}})_0^{H-1} \sim \mathcal{B}} \left[\sum_{t=0}^{H} \lambda^t \left(||\mathbf{z_t} - \text{sg}(h(\mathbf{s_t}))||_2^2 + \text{CE}(\hat{\mathbf{r}}_\mathbf{t}, \mathbf{r_{t+1}}) \right) \right] \\
\mathbf{z_{1:H}} = f(\mathbf{z_0}, \mathbf{a_{0:H-1}})
\end{aligned} \tag{6}$$

where sg is the stop-gradient operation, λ is a hyper-parameter that weights the near-prediction terms higher, CE is the cross-entropy loss of the two-hot distribution, and $\mathbf{z_{1:H}}$ is the chunked rollout by the chunked world dynamics f.

Value Learning. Similar to [29], although we adopt a chunked actor as the base policy, during inference we only execute the first step of the action chunk to ensure close-loop responsiveness to the environment. Therefore, different from the multi-step action Q function used in Q-Chunking[28], we employ a single-step action Q functions as the critic and update the critic with the classic 1-step TD error:

$$\begin{aligned}
\mathcal{L}_{\text{critics}} = \mathop{\mathbb{E}}_{(\mathbf{s_t}, \mathbf{a_t}, \mathbf{r_t}, \mathbf{s_{t+1}})_0^{H-1} \sim \mathcal{B}} \left[\sum_{t=0}^{H} \lambda^t \text{CE}(\hat{q}_t, q_t) \right] \\
q_t = \text{sg}(\mathbf{r_{t+1}} + Q'(\mathbf{z_{t+1}}, \pi_\theta(\mathbf{z_{t+1}}))),\ \mathbf{z_{t+1}} = h(\mathbf{s_{t+1}})
\end{aligned} \tag{7}$$

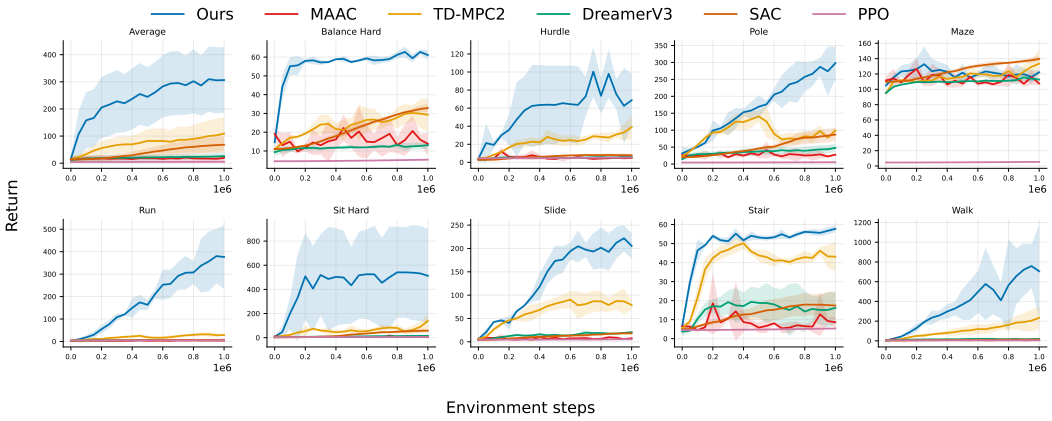


Figure 2: **Results in HumanoidBench (9 tasks shown).** Mean of 3 runs with different seeds. Shaded areas are 95% confidence intervals. CMPO achieves the best average performance and sample efficiency across baselines. Refer to Appendix C.4 for additional results.

where q_t is the TD target and Q' is the target Q function updated via exponential moving average.

Policy Learning. We use model-based value expansion [39] as our training-time policy optimization objective, which is closely related to the objective in Stochastic Value Gradient methods [22] and Model-Augmented Actor-Critic [23]. Since both the actor and the world model operate at the chunk level, the actor can optimize the single chunk-level rollout with ease, avoiding recurrent model unrolling and long-horizon BPTT. Instead of using a full H-step value expansion objective, which requires querying the actor again at the terminal imagined latent $\mathbf{z_{t+H}}$, we adopt an $(H-1)$-step surrogate. With SAC-style entropy regularization [8], the actor objective is:

$$\mathcal{L}_{\text{policy}} = \mathop{\mathbb{E}}_{\substack{\mathbf{s}\sim\mathcal{B} \\ \mathbf{a_{0:H-1}}\sim\pi}} \left[-\alpha \Big(\sum_{i=0}^{H-2} \gamma^i R(\mathbf{z_i}, \mathbf{a_i}) + \gamma^{H-1} Q(\mathbf{z_{H-1}}, \mathbf{a_{H-1}}) \Big) - \beta \mathcal{H}(\pi(\cdot|\mathbf{s})) \right] \tag{8}$$

$$\mathbf{z_{1:H}} = f(\mathbf{z_0}, \mathbf{a_{0:H-1}})$$

where $\mathcal{H}$ is the entropy of the action distribution, the chunked rollout $\mathbf{z_{1:H}}$ is inferred by the one-step invocation of the chunked world dynamics f and α, β are the hyper-parameters that control the degree of entropy regularization. We provide the pseudocode for CMPO in Appendix B.1 and the detailed architecture design in Appendix B.2.

4 Experiments

The goal of our empirical evaluation is to assess whether collaborative chunk-wise modeling can benefit efficient model-gradient-based policy optimization. In particular, we investigate whether CMPO can achieve competitive performance and sample efficiency compared with strong model-free and model-based RL baselines, while also evaluating its applicability to real-world robotic systems. Finally, we further analyze several key properties of CMPO, including optimization stability during training, the effects of different horizon settings, and its integration with test-time planning.

4.1 High-dimensional Humanoid Continuous Control

We evaluate our method on 19 selected challenging high-dimensional ($\mathcal{A} \in \mathbb{R}^{61}$) continuous control tasks from HumanoidBench [31]. These tasks require learning coordinated whole-body behaviors under large state and action spaces, which are suitable for evaluating sample efficiency, training stability, and long-horizon policy optimization of RL algorithms.

Baselines. We compare our method against both model-free and model-based RL algorithms. For model-free baselines, we include PPO [9] and SAC [8], two widely used continuous-control algorithms with strong empirical performance. For model-based baselines, we compare with TD-

MPC2 [18], a strong MPC-style model-based RL method that combines latent dynamics learning with decision-time trajectory optimization, and DreamerV3 [15], a representative imagination-based method that trains actors and critics through recurrent latent rollouts and backpropagate the expected returns through the model. We additionally include MAAC [23] as a gradient-based model-based RL baseline. This comparison allows us to evaluate whether our chunk-wise formulation can retain the benefits of model-based methods while avoiding the instable optimization of BPPT. We provide the more experimental setting details in the Appendix C.

Results. We present the averaged results across all 19 selected tasks in Figure 2 and demonstrate a subset of the results. The complete results are provided in Appendix C.4. The results show that CMPO achieves substantially better average performance and surpasses the baselines in the majority of the evaluated tasks. Additionally, CMPO demonstrates a significant advantage in sample efficiency. Furthermore, we compare the training wall-time of TD-MPC2 and CMPO. We benchmark the training wall-time of both methods on a single NVIDIA RTX 4090 under identical settings and without employing optimization techniques such as Torch Compile. The results is as shown in Table 1. Benefiting from one-step hypothetical rollout generation enabled by chunk-wise modeling, as well as eliminating iterative zeroth-order optimization during inference, CMPO requires only around 50% of the training time of TD-MPC2 under the same number of training steps while achieving approximately a 30× speedup in inference, further demonstrating its practical efficiency.

4.2 Quadruped Locomotion & Sim-to-Real Deployment

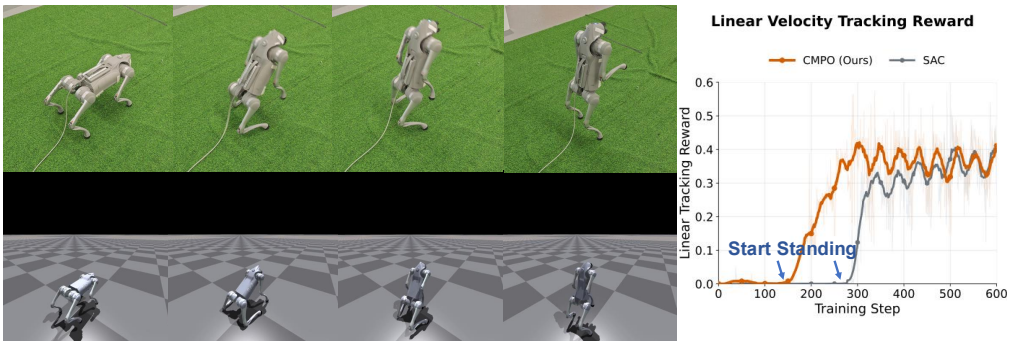


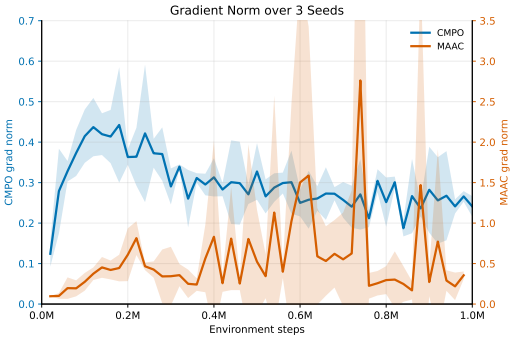
Figure 3: **Sim-to-real transfer of standing locomotion on Unitree Go2.**: **Left:** Representative snapshots of the learned standing locomotion policy in simulation and on the real robot. **Right:** Training curve of the standing linear-tracking reward in simulation.

	Walk		Run	
Wall-time	TD-MPC2	CMPO	TD-MPC2	CMPO
200k training steps (h) ↓	3.57	**1.74**	3.54	**1.72**
one inference step (ms) ↓	40.8	**1.4**	39.5	**1.3**

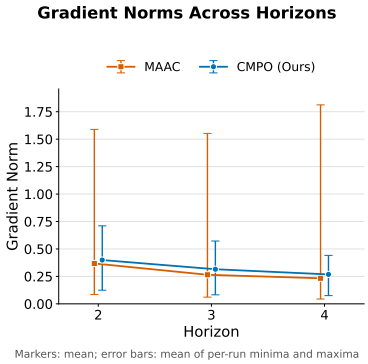
Table 1: **Training and inference Wall-time.**

One of the key advantages of CMPO is its ability to achieve strong performance with a direct-output inference pipeline which outperforms methods that rely on MPC-based test-time planning. This significantly reduces inference latency as shown in Table 1, enabling real-time responses to the dynamic environment, which is both beneficial to training efficiency and the real-world robotic deployment. Therefore, we further conduct sim-to-real deployment experiments on the Unitree Go2 quadruped robot platform. We evaluate SAC and CMPO under the same training and deployment pipeline. Figure 3 shows the sim-2-real evaluation of standing locomotion on Unitree Go2 and the standing linear velocity tracking results of SAC and CMPO. The reward is defined as the linear velocity tracking objective gated by the robot base height, such that the tracking reward is activated only when the base height exceeds a predefined threshold. In simulation, CMPO discovers standing substantially earlier and achieves higher tracking reward. On hardware, maintaining the bipedal posture for 20s succeeds in **7/10** trials for CMPO versus **4/10** for SAC. We use SAC because TD-MPC2's iterative test-time planning is not directly compatible with the same low-latency TorchScript feed-forward deployment interface. Full reward are provided in the Appendix C.3.

4.3 Discussion

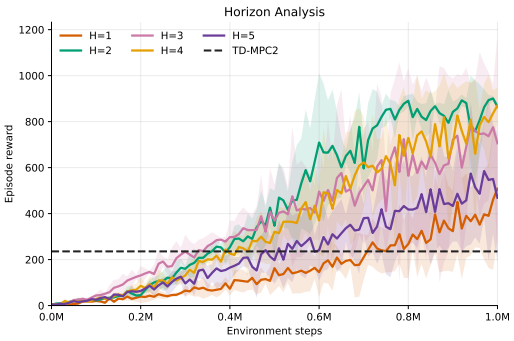


(a) **Gradient instability in MAAC.**

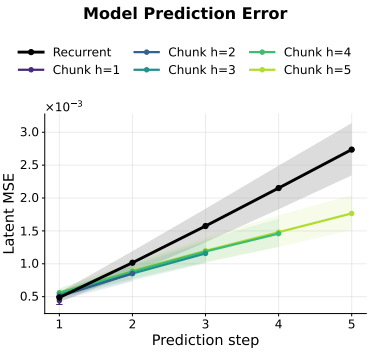


(b) **Gradient Norm across Horizons.**

Gradient instability in MAAC. One of our primary motivations is to employ chunk-wise modeling for both the policy and the world model to mitigate the optimization instability introduced by BPTT over long horizons. To validate this, we report the gradient norms during training for the classical single-step modeling approach MAAC and compare them with those of CMPO. As shown in Figure 4a, we find that MAAC exhibits highly oscillatory gradient norms, consistent with the characteristics of compositional gradient operators 4, while CMPO, benefiting from chunk-wise modeling, demonstrates substantially more stable optimization dynamics. We also highlight BPTT pathology is caused by recurrence and can already manifest at moderate horizons. As shown in Fig. 4b, MAAC exhibits larger gradient excursions, whereas CMPO maintains a narrower and more stable range with horizon increasing.



(a) **CMPO across Horizons.**



(b) **Model Prediction Error.**

CMPO with diverse horizon. Collaborative chunk-wise modeling avoids recursive model prediction error propagation. Therefore, as shown in Fig. 5b, the recurrent rollout error of single step model grows substantially faster with prediction step increasing, while chunk models consistently achieve lower latent prediction error. Also, the chunk-model error increases with horizon, explaining the degradation at larger horizons shown in Figure 5a. We further investigate the performance of CMPO under different horizon settings. The results in Figure 5a indicate that chunk-wise modeling with an appropriate horizon can substantially improve performance, with the best performance achieved at horizon = 2 and 4. In the main experiments, for a fair comparison with TD-MPC2, we consistently use horizon = 3 across all settings, which further demonstrates the strong performance of CMPO even without optimal horizon tuning. We observe that performance degrades when the horizon increases to 5, suggesting that prediction errors remain a limiting factor under longer-horizon rollouts. Overall, CMPO removes the recurrent temporal depth of model rollout/BPTT, but does not eliminate the intrinsic difficulty of direct long-horizon prediction, explaining why a moderate horzion is optimal.

CMPO without the chunk-wise model. Our key design lies in the collaborative chunk-wise modeling of both the policy and world dynamics model to enable one-step hypothetical rollout generation. A natural question is whether chunk-wise modeling remains effective when applied to only one component. We note that combining a chunk-wise model with a single-step policy is ill-posed, as future latent-state chunks cannot be properly grounded by single-step actions. Therefore, in Table 2, we only evaluate the setting with a chunk-wise policy and a single-step model (w/o CM). The results indicate that while the chunk-wise policy eliminates repeated policy calls, the recursive invocations required by the single-step model still lead to significant performance degradation, further highlighting the negative effects of pathological BPTT dynamics in model-gradient-based methods.

	Walk	Run	Stair	Slide
CMPO	**954.1**	629.2	60.9	265.8
w MPC	931.1	**864.5**	**71.9**	**391.2**
w/o CM	22.2	9.6	37.1	36.1

Table 2: Best average performance of variants with model prediction control (w MPC) and without the chunk-wise model (w/o CM).

CMPO with test-time model-based planning. To highlight the effectiveness of chunk-wise modeling itself, we do not adopt MPC-based test-time planning in the main experiments. Since chunk-wise modeling and MPC are orthogonal design choices, we further combine CMPO with the same MPPI planner used in TD-MPC2 and report the results in Table 2. We observe that MPC improves performance on unsaturated tasks, likely due to the additional exploration induced by MPPI's sampling-based zeroth-order optimization. Meanwhile, on simpler tasks, the gradient information provided by chunk-wise modeling alone is sufficient to reach saturated performance.

5 Conclusion

We propose a collaborative chunk-wise modeling framework for Model-Based Reinforcement Learning, in which both the actor and the world dynamics model generate chunk-level foresight predictions. This chunk-wise design enables the generation of hypothetical future rollouts without recurrent model invocation, thereby reducing the computational overhead introduced by recursive calls. Moreover, in model-gradient-based training, it effectively stabilizes the optimization under long horizons by mitigating gradient oscillations and avoiding the pathological BPTT dynamics associated with single-step models. The simulation results on high-dimensional humanoid continuous control tasks demonstrate that our method, relying solely on model-gradient-based optimization, surpasses both model-based methods that employ test-time planning and strong model-free baselines. Finally, the real-world demonstrations of bipedal walking on the Unitree Go2 quadruped robot further validate its practical efficiency.

Limitation

In this work, we primarily focus on building a stable model-gradient-based method on top of TD-MPC2, and most of the model architecture is inherited from the design of TD-MPC2. Exploring network architectures that are more suitable for chunk-wise modeling in reinforcement learning remains an open question. In addition, to ensure closed-loop responsiveness in dynamic environments, our chunk policy executes only the first action of each predicted chunk. However, for less dynamic environments, this design implies that the remaining actions within the chunk are discarded and therefore underutilized. While some prior work [40–42] in imitation learning has explored dynamic action horizons, this direction remains relatively underexplored in reinforcement learning. Therefore, an important future direction is how to effectively integrate our collaborative chunk-wise modeling framework with dynamic horizon execution.

Acknowledgments

This work was supported in part by Shanghai Municipal Science and Technology Major Project (2025SHZDZX025G14), National Natural Science Foundation of China (U25A20442, 62306175).

References

[1] D. Hoeller, N. Rudin, D. Sako, and M. Hutter. Anymal parkour: Learning agile navigation for quadrupedal robots. *Science Robotics*, 9(88):eadi7566, 2024.

[2] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 9(89):eadi9579, 2024.

[3] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. Van Wyk, A. Zhurkevich, B. Sundaralingam, et al. Dextreme: Transfer of agile in-hand manipulation from simulation to reality. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5977–5984. IEEE, 2023.

[4] W. Ye, S. Liu, T. Kurutach, P. Abbeel, and Y. Gao. Mastering atari games with limited data. *Advances in neural information processing systems*, 34:25476–25488, 2021.

[5] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.

[6] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[7] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[11] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. P. Wierstra. Continuous control with deep reinforcement learning, Sept. 15 2020. US Patent 10,776,692.

[12] R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.

[13] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

[14] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Y. Zou, S. Levine, C. Finn, and T. Ma. Mopo: Model-based offline policy optimization. *Advances in neural information processing systems*, 33: 14129–14142, 2020.

[15] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[16] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

[17] N. Hansen, X. Wang, and H. Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.

[18] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, volume 2024, pages 47376–47405, 2024.

[19] M. Okada, N. Kosaka, and T. Taniguchi. Planet of the bayesians: Reconsidering and improving deep planning network by incorporating bayesian inference. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5611–5618. IEEE, 2020.

[20] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in neural information processing systems*, 31, 2018.

[21] Y. Wang, H. Guo, S. Wang, L. Qian, and X. Lan. Bootstrapped model predictive control. *arXiv preprint arXiv:2503.18871*, 2025.

[22] B. Amos, S. Stanton, D. Yarats, and A. G. Wilson. On the model-based stochastic value gradient for continuous reinforcement learning. In *Learning for Dynamics and Control*, pages 6–20. PMLR, 2021.

[23] I. Clavera, V. Fu, and P. Abbeel. Model-augmented actor-critic: Backpropagating through paths. *arXiv preprint arXiv:2005.08068*, 2020.

[24] J. Amigo, R. Khorrambakht, E. Chane-Sane, N. Mansard, and L. Righetti. First order model-based rl through decoupled backpropagation. *arXiv preprint arXiv:2509.00215*, 2025.

[25] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[26] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[27] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[28] Q. Li, Z. P. Zhou, and S. Levine. Reinforcement learning with action chunking. *Advances in Neural Information Processing Systems*, 38:55518–55553, 2026.

[29] Q. Li, S. Park, and S. Levine. Decoupled q-chunking. *arXiv preprint arXiv:2512.10926*, 2025.

[30] K. Park, S. Park, Y. Lee, and S. Levine. Scalable offline model-based rl with action chunks. *arXiv preprint arXiv:2512.08108*, 2025.

[31] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.

[32] T. Kurutach, I. Clavera, Y. Duan, A. Tamar, and P. Abbeel. Model-ensemble trust-region policy optimization. *arXiv preprint arXiv:1802.10592*, 2018.

[33] Y. Luo, H. Xu, Y. Li, Y. Tian, T. Darrell, and T. Ma. Algorithmic framework for model-based deep reinforcement learning with theoretical guarantees. *arXiv preprint arXiv:1807.03858*, 2018.

[34] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims. Morel: Model-based offline reinforcement learning. *Advances in neural information processing systems*, 33:21810–21823, 2020.

[35] T. Yu, A. Kumar, R. Rafailov, A. Rajeswaran, S. Levine, and C. Finn. Combo: Conservative offline model-based policy optimization. *Advances in neural information processing systems*, 34:28954–28967, 2021.

[36] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 7559–7566. IEEE, 2018.

[37] H. Lin, P. Wang, J. Schneider, and G. Shi. Td-m(pc)^2: Improving temporal difference mpc through policy constraint. *arXiv preprint arXiv:2502.03550*, 2025.

[38] N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa. Learning continuous control policies by stochastic value gradients. *Advances in neural information processing systems*, 28, 2015.

[39] V. Feinberg, A. Wan, I. Stoica, M. I. Jordan, J. E. Gonzalez, and S. Levine. Model-based value estimation for efficient model-free reinforcement learning. *arXiv preprint arXiv:1803.00101*, 2018.

[40] Y. Liang, X. Wang, K. Wang, S. Wang, X. Peng, H. Chen, D. K. H. Chua, and P. Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models. *arXiv preprint arXiv:2604.04161*, 2026.

[41] Q. Wen, H. Zhu, Y. Zhang, L. Xia, B. Gao, and Z. Li. Adaptive action chunking for robotic imitation learning. *Biomimetics*, 11(5):316, 2026.

[42] W. Zhang, T. Hu, H. Zhang, Y. Qiao, Y. Qin, Y. Li, J. Liu, T. Kong, L. Liu, and X. Ma. Chain-of-action: Trajectory autoregressive modeling for robotic manipulation. *Advances in Neural Information Processing Systems*, 38:106070–106094, 2026.

A Proof

Proposition 1 (Compounding error of recurrent model rollouts)**.** *Consider a deterministic latent dynamics function $f : \mathcal{Z} \times \mathcal{A} \to \mathcal{Z}$ and a learned dynamics model $\hat{f}_\phi : \mathcal{Z} \times \mathcal{A} \to \mathcal{Z}$. Given the same initial latent state $z_t = \hat{z}_t$ and the same open-loop action sequence $a_{t:t+H-1}$, define the true and imagined rollouts as $z_{t+k+1} = f(z_{t+k}, a_{t+k})$, $\hat{z}_{t+k+1} = \hat{f}_\phi(\hat{z}_{t+k}, a_{t+k})$, for $k = 0, \dots, H-1$. Assume that the learned model has uniformly bounded one-step prediction error*

$$\|\hat{f}_\phi(z, a) - f(z, a)\| \le \epsilon, \qquad \forall (z, a) \in \mathcal{D},$$

where $\mathcal{D}$ contains the states and actions visited by the rollouts. Assume further that the true dynamics f is L_f-Lipschitz with respect to the latent state, i.e.,

$$\|f(z, a) - f(z', a)\| \le L_f \|z - z'\|, \qquad \forall z, z' \in \mathcal{Z},\ \forall a \in \mathcal{A}.$$

Then, for any rollout step $k \ge 1$, the imagined rollout error satisfies

$$\|\hat{z}_{t+k} - z_{t+k}\| \le \epsilon \sum_{i=0}^{k-1} L_f^i.$$

Proof. Let

$$e_k = \|\hat{z}_{t+k} - z_{t+k}\|.$$

Since both rollouts start from the same initial latent state, we have $e_0 = 0$. For the next-step error,

$$\begin{aligned} e_{k+1} &= \|\hat{z}_{t+k+1} - z_{t+k+1}\| \\ &= \|\hat{f}_\phi(\hat{z}_{t+k}, a_{t+k}) - f(z_{t+k}, a_{t+k})\|. \end{aligned}$$

By adding and subtracting $f(\hat{z}_{t+k}, a_{t+k})$, we obtain

$$\begin{aligned} e_{k+1} &= \|\hat{f}_\phi(\hat{z}_{t+k}, a_{t+k}) - f(\hat{z}_{t+k}, a_{t+k}) \\ &\quad + f(\hat{z}_{t+k}, a_{t+k}) - f(z_{t+k}, a_{t+k})\| \\ &\leq \|\hat{f}_\phi(\hat{z}_{t+k}, a_{t+k}) - f(\hat{z}_{t+k}, a_{t+k})\| \\ &\quad + \|f(\hat{z}_{t+k}, a_{t+k}) - f(z_{t+k}, a_{t+k})\|. \end{aligned}$$

Using the bounded one-step model error and the Lipschitz continuity of f, we have

$$e_{k+1} \leq \epsilon + L_f e_k.$$

Unrolling this recursion gives

$$\begin{aligned} e_k &\leq \epsilon + L_f\epsilon + L_f^2\epsilon + \cdots + L_f^{k-1}\epsilon \\ &= \epsilon \sum_{i=0}^{k-1} L_f^i. \end{aligned}$$

Equivalently, when $L_f \neq 1$, $\|\hat{z}_{t+k} - z_{t+k}\| \leq \epsilon \frac{L_f^k - 1}{L_f - 1}$, and when $L_f = 1$, $\|\hat{z}_{t+k} - z_{t+k}\| \leq k\epsilon$. This completes the proof. □

B Implementation Details

B.1 Pseudocode for CMPO

Algorithm 1: Chunk-wise Model-based Policy Optimization (CMPO)

Require: Initialize chunk policy π, chunk world model f, critics Q_ψ, targte critics $Q_{\psi'}$,state encoder h and reward model R randomly.

while *not converged* **do**
  // Collect experiences
  for *step* $t = 0 \ldots T$ **do**
    Generate chunked action distribution $\pi_{t:t+C} \leftarrow \pi(\cdot \mid h(\mathbf{s}_t), p_\theta)$;
    Sample chunked action sequence $\mathbf{a}_{t:t+C} \sim \pi_{t:t+C}$;
    Execute the first action $(\mathbf{s}_{t+1}, r_t) \leftarrow \text{env.step}(\mathbf{a}_t)$;
    Store the trainsition $\mathcal{B} \leftarrow \mathcal{B} \cup (\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$;
  // Update networks
  for *num updates per episode* **do**
    Sample chunked batch $\{\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}, \pi_t\}_{t:t+H} \sim \mathcal{B}$;
    Update h, f, R via Equation 6;
    Update Q_ψ via Equation 7;
    Update π_θ via Equation 8;
    Update Q_{ϕ^-} via EMA.

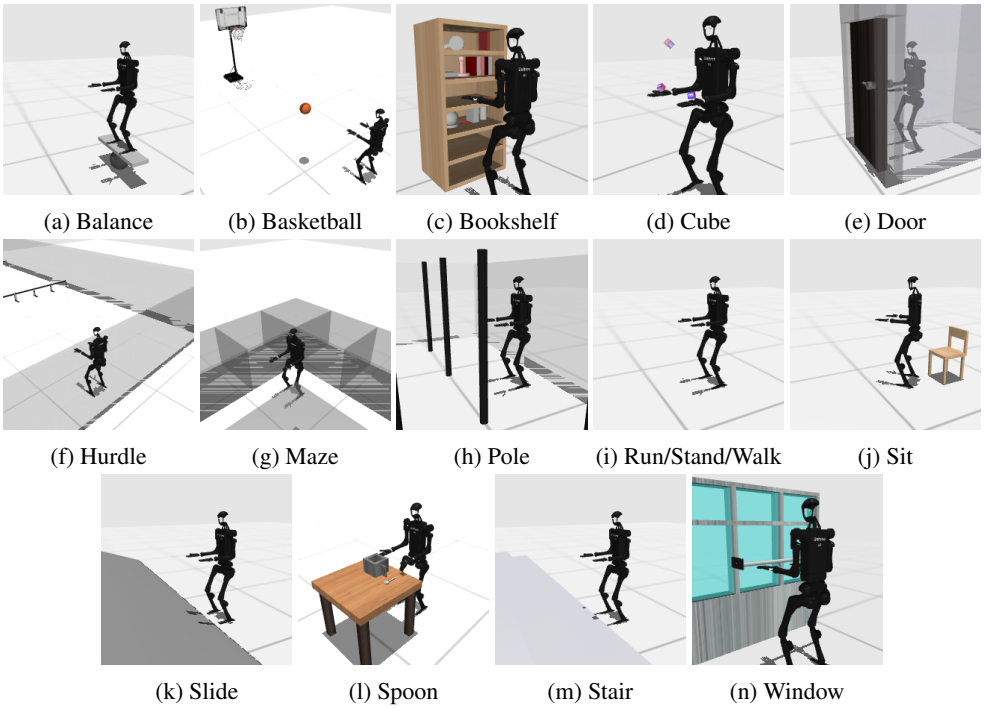
B.2 Network Settings

We retain most of the network architecture and hyperparameter settings from TD-MPC2 to ensure that the performance gains of CMPO arise solely from our collaborative chunk-wise modeling. Our chunk-wise policy and dynamics model are built upon the MLP-based single-step policy/model architecture of TD-MPC2. Concretely, we extend the output size of the single-step policy from the original $action\ dim$ to $action\ dim \times horizon$. For the chunk-wise dynamics model, the input dimension is expanded from $latent\ dim + action\ dim$ to $latent\ dim + horizon \times action\ dim$, and the output dimension is correspondingly extended from $latent\ dim$ to $latent\ dim \times horizon$. Such a design introduces only minimal modifications to the original TD-MPC2 architecture, ensuring that the observed performance gains primarily stem from collaborative chunk-wise modeling rather than architectural differences.

C Experimental Details

C.1 Simulation Benchmark Overview

We evaluate our method in 19 selected high-dimensional($\mathcal{A} \in \mathbb{R}^{61}$) whole-body tasks with a Unitree H1hand robot which are originally from HumanoidBench [31]. These 19 selected tasks cover a diverse set of high-dimensional humanoid control challenges, including basic locomotion such as *walking*, *running*, and *standing*; terrain traversal tasks such as *stairs*, *slides*, *hurdles*, *mazes*, and *poles*; and posture control tasks like *balancing*; as well as object- and scene-interaction tasks such as *door opening*, *cube manipulation*, *bookshelf interaction*, *basketball*, *window manipulation*, and *spoon handling*. Together, these tasks evaluate both whole-body locomotion and fine-grained manipulation capabilities, providing a comprehensive benchmark for assessing the robustness and generality of our method.Detailed reward specifications can be found in the original HumanoidBench paper. We additionally provide visualizations for the selected tasks.



(a) Balance (b) Basketball (c) Bookshelf (d) Cube (e) Door

(f) Hurdle (g) Maze (h) Pole (i) Run/Stand/Walk (j) Sit

(k) Slide (l) Spoon (m) Stair (n) Window

Figure 6: Visualization of the selected tasks.

C.2 Simulation Baseline Specification

For baseline comparisons, we directly adopt the reported results of SAC, PPO, DreamerV3, and TD-MPC2 from the original HumanoidBench benchmark to ensure consistency with the official evaluation protocol. In addition, we implement MAAC-style policy updates on top of the TD-MPC2 architecture as an additional baseline, enabling a more controlled comparison between classical single-step model-gradient optimization and our collaborative chunk-wise modeling framework.

C.3 Real World Experiment Setup

To evaluate the real-world performance and sim-to-real transferability of our method, we conduct experiments on a bipedal locomotion task using a Unitree Go2 quadruped. We first train the locomotion policy in Isaac Gym and subsequently deploy the learned policy on the real robot. The policy controls all 12 actuated joints through residual joint-position commands, which are tracked

by a PD controller. The reward function encourages the robot to maintain an upright bipedal posture while tracking commanded linear and angular velocities, together with regularization terms on joint acceleration, power consumption, action variation, collisions, and hip motion. During simulation training, the robot learns to stand and locomote using its hind legs, and the resulting policy is transferred to the physical Go2 platform for real-world evaluation.

C.3.1 Reward Design

Here, $\mathbf{v}^b$ and $\boldsymbol{\omega}^b$ denote the base linear and angular velocities expressed in the robot body frame, respectively; $\mathbf{g}^{\mathrm{proj}}$ is the gravity vector projected into the body frame; $\mathbf{q}$, $\dot{\mathbf{q}}$, and $\boldsymbol{\tau}$ denote joint position, velocity, and torque; and $\mathbf{a}_t$ denotes the policy action. $\mathcal{C}$ contains the thigh, calf, hip, and base links, while $\mathcal{H}$ denotes the four hip joints. We use $\sigma = 0.25$ for velocity tracking. The upright reference configuration is $q^{\mathrm{ref}}_{\mathrm{hip}} = 0$, $q^{\mathrm{ref}}_{\mathrm{thigh}} = 1.8$, and $q^{\mathrm{ref}}_{\mathrm{calf}} = -1.1$ rad for each leg. The total reward is computed as $r_t = \sum_i w_i r_i$. Here, $\mathbf{1}_{\mathrm{up}} = \mathbf{1}[-g^{\mathrm{proj}}_x > 0.9]$ indicates that the robot is sufficiently upright. The listed weights correspond to the configured reward scales.

Table 3: **Reward terms for bipedal Go2 locomotion.**

Reward term	**Definition**	**Weight** w_i
Linear velocity tracking	$\exp\left(-\dfrac{(c_x + v^b_z)^2 + (c_y - v^b_y)^2}{\sigma}\right)\mathbf{1}_{\mathrm{up}}$	2.0
Angular velocity tracking	$\exp\left(-\dfrac{(c_{\mathrm{yaw}} - \omega^b_x)^2}{\sigma}\right)\mathbf{1}_{\mathrm{up}}$	2.0
DOF acceleration	$\sum_j \left(\dfrac{\dot{q}_{t-1,j} - \dot{q}_{t,j}}{\Delta t}\right)^2$	-2.5×10^{-7}
Joint power	$\sum_j \lvert\dot{q}_j\rvert \lvert\tau_j\rvert$	-2.0×10^{-5}
Base height	$\exp\left(-\dfrac{(h - h^\star)^2}{0.25}\right), \quad h^\star = 0.55\,\mathrm{m}$	8.0
Collision	$\sum_{k\in\mathcal{C}} \mathbf{1}[\lVert\mathbf{f}_k\rVert_2 > 0.1]$	-2.0
Action rate	$\lVert\mathbf{a}_t - \mathbf{a}_{t-1}\rVert_2^2$	-0.01
Action smoothness	$\lVert\mathbf{a}_t - 2\mathbf{a}_{t-1} + \mathbf{a}_{t-2}\rVert_2^2$	-0.01
Upright orientation	$-g^{\mathrm{proj}}_x$	2.0
Upright reference pose	$\exp\left(-\dfrac{\frac{1}{N_q}\lVert\mathbf{q} - \mathbf{q}^{\mathrm{ref}}\rVert_2^2}{0.25}\right)\mathbf{1}_{\mathrm{up}}$	2.0
Hip regularization	$\sum_{j\in\mathcal{H}} \lvert a_j\rvert$	-0.2

C.4 Complete Simulation Results

Figure 7 presents the complete results across all 19 selected tasks in the HumanoidBench.

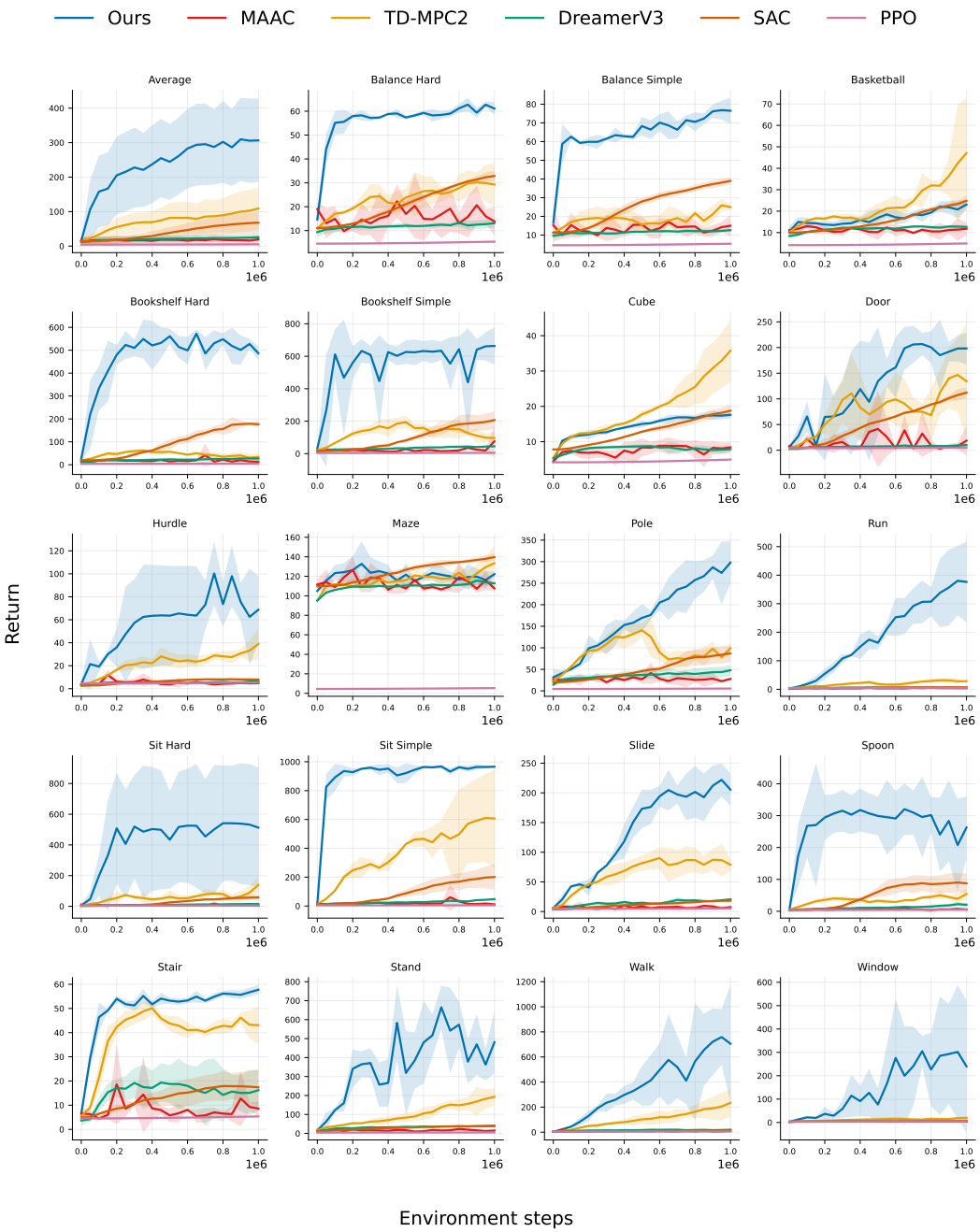


Figure 7: **All results in Humanoid Benchmark**. The reward curves of 19 tasks in Humanoid Bench. Mean of 3 runs with different seeds. Shaded areas are 95% confidence intervals.